

3. Precise typing of pattern matching (II)

Boolean type connectives are needed to type pattern matching:

`match e with p1 -> e1 | p2 -> e2`

Suppose that $e : T$

- To infer the type T_1 of e_1 we need $T \wedge \{p_1\}$;
- To infer the type T_2 of e_2 we need $(T \setminus \{p_1\}) \wedge \{p_2\}$; ($S \setminus T$ stands for $S \wedge \neg T$)
- The type of the match expression is $T_1 \vee T_2$.
- Pattern matching is exhaustive if $T \leq \{p_1\} \vee \{p_2\}$.

Formally:

$$\frac{\begin{array}{c} \text{[Match]} \\ \Gamma \vdash e : T \quad \Gamma, T \wedge \{p_1\} / p_1 \vdash e_1 : T_1 \quad \Gamma, T \setminus \{p_1\} / p_2 \vdash e_2 : T_2 \end{array}}{\Gamma \vdash \text{match } e \text{ with } p_1 \rightarrow e_1 \mid p_2 \rightarrow e_2 : T_1 \vee T_2} \quad (T \leq \{p_1\} \vee \{p_2\})$$

where T/p is the type for the capture variables in p when it is matched against values in T .